LAB 03

System Performance Evaluation Using Synthetic Benchmarking

Ackermann's Function and Matrix Multiplication

System Configuration and Performance Evaluation Lab
Course No: 0613-310

Ekramul Haque Amin
Lecturer, AI and Data Science
Dhaka International University (DIU)

From Lab 02 to Lab 03

Lab 02 showed

Under load, CPU time stayed about the same, but wall time grew, because the program had to wait.

Today we ask

Do different kinds of programs react to load in the same way?

We compare a recursive benchmark with a numerical benchmark

Today’s Objectives

1. Understand what a synthetic benchmark is
2. Implement Ackermann’s function as a recursive benchmark
3. Implement matrix multiplication as a numerical benchmark
4. Measure CPU time and wall time under Normal and Loaded conditions
5. Find out which benchmark is more sensitive to load

> **Deliverable: Lab Report 03**

What Is a Synthetic Benchmark?

An artificial program, written only for testing, that stresses one part of the system.

Benchmark	What it stresses
Whetstone	Floating-point arithmetic
Dhrystone	Integer and string operations
LINPACK	Solving linear equations (matrix math)
Ackermann's function	Function calls and recursion

Today’s Two Benchmarks

TASK 1 · RECURSIVE

Ackermann’s Function

- Millions of function calls
- Very deep call stack
- Almost no arithmetic

TASK 2 · NUMERICAL

Matrix Multiplication

- Millions of multiply-adds
- Floating-point arithmetic
- Reads large arrays from memory

Two very different workloads, tested under the same two conditions

Ackermann's Function: Three Rules

1	If m = 0	`A(0, n) = n + 1`
2	If n = 0	`A(m, 0) = A(m-1, 1)`
3	Otherwise	`A(m, n) = A(m-1, A(m, n-1))`

Rule 3 calls A inside A: that is what makes the calls explode

Tracing A(1, 2)

Step 1 / 6 · Calls: 1

Call stack

Newest call on top

`A(1, 2) · running`

```
  1. m = 0:     A(0, n) = n + 1
  2. n = 0:     A(m, 0) = A(m-1, 1)
> 3. otherwise: A(m, n) = A(m-1, A(m, n-1))
```

m = 1, n = 2: use rule 3

A(1, 2) = A(0, A(1, 1)). Before it can finish, it needs A(1, 1). So it calls A(1, 1) and waits.

Tracing A(1, 2)

Step 2 / 6 · Calls: 2

Call stack

Newest call on top

```
A(1, 1) · running
```

```
A(1, 2) · waits for A(1, 1)
```

```
  1. m = 0:      A(0, n) = n + 1
  2. n = 0:      A(m, 0) = A(m-1, 1)
> 3. otherwise: A(m, n) = A(m-1, A(m, n-1))
```

A(1, 1): rule 3 again

A(1, 1) = A(0, A(1, 0)). It needs A(1, 0) first, so it calls A(1, 0) and waits. The stack grows.

Tracing A(1, 2)

Step 3 / 6 · Calls: 3

Call stack

Newest call on top

- `A(1, 0) · running`
- `A(1, 1) · waits for A(1, 0)`
- `A(1, 2) · waits for A(1, 1)`

```
  1. m = 0:     A(0, n) = n + 1
> 2. n = 0:     A(m, 0) = A(m-1, 1)
  3. otherwise: A(m, n) = A(m-1, A(m, n-1))
```

A(1, 0): n = 0, so use rule 2

A(1, 0) = A(0, 1). It calls A(0, 1) and waits. Three calls are now waiting on the stack.

Tracing A(1, 2)

Step 4 / 6 · Calls: 4

Call stack

Newest call on top

- **A(0, 1) · returns 2**
- A(1, 0) · waits for A(0, 1)
- A(1, 1) · waits for A(1, 0)
- A(1, 2) · waits for A(1, 1)

```
> 1. m = 0:      A(0, n) = n + 1
  2. n = 0:      A(m, 0) = A(m-1, 1)
  3. otherwise:  A(m, n) = A(m-1, A(m, n-1))
```

A(0, 1): m = 0, rule 1 gives 2

The first real answer! This is the deepest point: 4 calls on the stack at once. Now answers flow back down: A(1, 0) = 2.

Tracing A(1, 2)

Step 5 / 6 · Calls: 5

Call stack

Newest call on top

- `A(0, 2) · returns 3`
- `A(1, 1) · waits for A(0, 2)`
- `A(1, 2) · waits for A(1, 1)`

```
> 1. m = 0:      A(0, n) = n + 1
  2. n = 0:      A(m, 0) = A(m-1, 1)
  3. otherwise:  A(m, n) = A(m-1, A(m, n-1))
```

A(1, 1) can now continue

A(1, 0) returned 2, so A(1, 1) = A(0, 2). That is a new call. Rule 1 gives 3, so A(1, 1) = 3.

Tracing A(1, 2)

Step 6 / 6 · Calls: 6

Call stack

Newest call on top

```
A(0, 3) · returns 4
```

```
A(1, 2) · waits for A(0, 3)
```

```
> 1. m = 0:     A(0, n) = n + 1
  2. n = 0:     A(m, 0) = A(m-1, 1)
  3. otherwise: A(m, n) = A(m-1, A(m, n-1))
```

Result: A(1, 2) = 4

A(1, 1) returned 3, so A(1, 2) = A(0, 3) = 4. The answer is a small number, but it took 6 calls and a stack 4 deep.

Small Inputs, Huge Work

Call	Answer	Function calls	Stack depth
A(1, 2)	4	6	4
A(2, 2)	7	27	8
A(3, 3)	61	2,432	63
A(3, 5)	253	42,438	255
A(3, 8)	2,045	2,785,999	2,047
A(3, 9) ← ours	4,093	11,164,370	4,095

Each +1 in n makes about 4× more calls

Python’s Recursion Limit

Problem

```
>>> ackermann(3, 9)
Traceback (most recent call last):
  ...
RecursionError: maximum
recursion depth exceeded
```

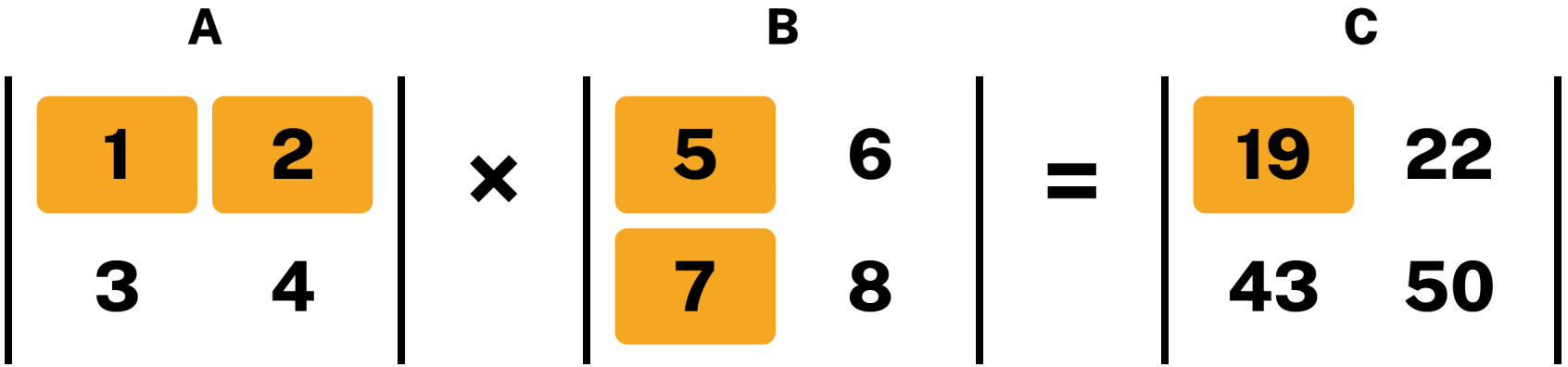
Fix

```
import sys

# allow deeper recursion
sys.setrecursionlimit(10_000)
```

Python allows about 1,000 calls deep by default. A(3, 9) needs 4,095.

Matrix Multiplication: How It Works



`c[0][0] = 1 × 5 + 2 × 7 = 19`

Each cell of C = row of A × column of B, multiplied pair by pair, then added

How Much Work? n^3

Each of the n × n cells needs n multiply-adds, so the total is n × n × n = n^3.

Matrix size n	Multiply-adds (n^3)	Our test (CPU s)
100 × 100	1,000,000	0.05
200 × 200	8,000,000	0.40
250 × 250 ← ours	15,625,000	0.82

Double n → 2^3 = 8× the work (0.05 s → 0.40 s)

Two Benchmarks Side by Side

Aspect	Ackermann A(3, 9)	Matrix 250 × 250
Kind	Recursive	Numerical
Main work	11.2 million function calls	15.6 million multiply-adds
Stresses	Call overhead, stack memory	Floating-point math, memory reads
Data used	Two small integers	Three matrices, 187,500 numbers
Real-world cousin	Recursive code, compilers	Scientific computing, machine learning

Code: Ackermann's Function

```
import sys
sys.setrecursionlimit(10_000)

def ackermann(m, n):
    if m == 0:
        return n + 1                              # rule 1
    if n == 0:
        return ackermann(m - 1, 1)                # rule 2
    return ackermann(m - 1, ackermann(m, n - 1))  # rule 3
```

The code is the three rules, written almost word for word

Code: Matrix Multiplication

```
def matrix_multiply(a, b):
    n = len(a)
    c = [[0.0] * n for _ in range(n)]
    for i in range(n):                      # each row of A
        for j in range(n):                  # each column of B
            total = 0.0
            for k in range(n):              # pair by pair
                total += a[i][k] * b[k][j]
            c[i][j] = total
    return c
```

Three nested loops of n → n^3 multiply-adds

Code: Timing and Running Safely

```
def measure(func, *args):                    # same idea as Lab 02
    wall_start = time.perf_counter()
    cpu_start = time.process_time()
    result = func(*args)
    cpu_time = time.process_time() - cpu_start
    wall_time = time.perf_counter() - wall_start
    return result, cpu_time, wall_time

# run main() in a thread with a 64 MB stack
threading.stack_size(64 * 1024 * 1024)
worker = threading.Thread(target=main)
worker.start()
worker.join()
```

Sample Output: Normal Condition

```
TASK 1: ACKERMANN(3, 9)  (recursive)
Run       CPU Time (s)     Wall Time (s)
-----------------------------------------
1               1.0393            1.0541
2               1.0575            1.0692
3               1.0830            1.1039
4               1.0861            1.1072
5               1.0575            1.0673
-----------------------------------------
CPU Time   Average = 1.0647 s  SD = 0.0196 s  CV = 1.84 %
Wall Time  Average = 1.0804 s  SD = 0.0238 s  CV = 2.20 %
Average waiting time (Wall - CPU) = 0.0157 s
Check: Ackermann(3, 9) = 4093
```

Task 2 prints the same table for the matrix multiplication

Our Test Run: Normal vs Loaded

Benchmark	Normal CPU	Normal Wall	Loaded CPU	Loaded Wall	Wall slowdown
Ackermann	1.06 s	1.08 s	1.09 s	2.75 s	2.5×
Matrix	0.82 s	0.84 s	0.86 s	2.20 s	2.6×

- CPU time hardly changed for either benchmark
- Wall time grew about 2.5× for both
- On this computer, both were about equally sensitive

Your results may differ. Explain what your numbers show.

Step A: Normal System Condition

1. Close all unnecessary applications
2. Check Task Manager: CPU usage should be low
3. Run: python synthetic_benchmark.py
4. Do not touch the computer until it finishes
5. Screenshot the output; record the Normal rows of Table 1 (Ackermann) and Table 2 (Matrix)

> **One run of the program gives 5 runs of both benchmarks**

Step B: Loaded System Condition

1. Create heavy load, the same way as Lab 02. Open several terminals and in each run:

```
python -c "while True: pass"
```

1. Use one more terminal than the number of logical cores
2. Run the same program again; record the Loaded rows of Tables 1 and 2
3. Press Ctrl + C in every terminal when finished

Recording the Results

Tables 1 and 2

For each benchmark, copy runs 1–5 and the average for Normal and for Loaded, with CPU time and wall time.

Performance comparison

Benchmark	Normal	Loaded
Ackermann avg. CPU time		
Ackermann avg. wall time		
Matrix avg. CPU time		
Matrix avg. wall time		

Which Is More Sensitive to Load?

Slowdown = Loaded avg. wall time ÷ Normal avg. wall time

Benchmark	Our test	Slowdown
Ackermann	2.75 s ÷ 1.08 s	2.5×
Matrix	2.20 s ÷ 0.84 s	2.6×

Bigger slowdown = more sensitive to load

Analysis Questions

1. Compare benchmark performance under Normal and Loaded conditions
2. Compare the Ackermann and matrix multiplication benchmarks
3. How did increased load affect CPU time and wall time?
4. Which benchmark was more sensitive to load? Explain
5. Were the results consistent across all runs? Use the CV

> **Answer with your own numbers, not general statements**

Common Mistakes to Avoid

Mistake	Fix
Using A(3, 5)	Too fast to time; use A(3, 9)
RecursionError	sys.setrecursionlimit(10_000)
Creating the matrices inside the timed part	Create them once; time only the multiply
Trying A(4, 1)	Far too many calls to finish; stay at m = 3
Different sizes for Normal and Loaded	Same A(3, 9) and 250 × 250 for both

Key Takeaway

Different benchmarks stress different parts of a computer, so one benchmark never tells the whole story.

Under load: CPU time stays about the same · Wall time grows

Next lab: Profiling: finding where a program spends its time with cProfile and pstats